Sorting algorithms

8 min read Updated 16 Jul 2026 Topic 58 of 77

23A. Sorting Algorithms (12 problems)

Pattern recognition: "Sort these items", "Order by", "Kth largest/smallest", "Custom ordering"

23.1 Bubble Sort

JAVASCRIPT Copy

```javascript
function bubbleSort(arr) {
  const n = arr.length;
  for (let i = 0; i < n - 1; i++) {
    let swapped = false;
    for (let j = 0; j < n - 1 - i; j++) {
      if (arr[j] > arr[j + 1]) {
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
        swapped = true;
      }
    }
    if (!swapped) break;
  }
  return arr;
}
```

Time: O(n²) avg/worst, O(n) best. **Space:** O(1). **Stable:** Yes.

JAVASCRIPT Copy

```javascript
function selectionSort(arr) {
  for (let i = 0; i < arr.length - 1; i++) {
    let minIdx = i;
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[j] < arr[minIdx]) minIdx = j;
    }
    if (minIdx !== i) [arr[i], arr[minIdx]] = [arr[minIdx], arr[i]];
  }
  return arr;
}
```

Time: O(n²) always. **Space:** O(1). **Stable:** No.

23.3 Insertion Sort

JAVASCRIPT Copy

```javascript
function insertionSort(arr) {
  for (let i = 1; i < arr.length; i++) {
    const key = arr[i];
    let j = i - 1;
    while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; j--; }
    arr[j + 1] = key;
  }
  return arr;
}
```

Time: O(n²) worst, O(n) best (nearly sorted). **Space:** O(1). **Stable:** Yes.

Frontend connection: Efficient for nearly-sorted data — when a user drags one item in a list, insertion sort re-sorts in O(n).

JAVASCRIPT Copy

```
function mergeSort(arr) {
  if (arr.length ≤ 1) return arr;
  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));
  return merge(left, right);
}

function merge(left, right) {
  const result = [];
  let i = 0, j = 0;
  while (i < left.length && j < right.length) {
    if (left[i] ≤ right[j]) result.push(left[i++]);
    else result.push(right[j++]);
  }
  return [...result, ...left.slice(i), ...right.slice(j)];
}
```

Time: O(n log n) always. **Space:** O(n). **Stable:** Yes.

23.5 Quick Sort

JAVASCRIPT Copy

```
function quickSort(arr, lo = 0, hi = arr.length - 1) {
  if (lo ≥ hi) return arr;
  const pivotIdx = partition(arr, lo, hi);
  quickSort(arr, lo, pivotIdx - 1);
  quickSort(arr, pivotIdx + 1, hi);
  return arr;
}

function partition(arr, lo, hi) {
  const pivot = arr[hi];
  let i = lo;
```

```
  [arr[i], arr[hi]] = [arr[hi], arr[i]];
  return i;
}
```

Time: O(n log n) avg, O(n²) worst. **Space:** O(log n). **Stable:** No.

23.6 Heap Sort

JAVASCRIPT Copy

```
function heapSort(arr) {
  const n = arr.length;
  for (let i = Math.floor(n / 2) - 1; i ≥ 0; i--) heapify(arr, n, i);
  for (let i = n - 1; i > 0; i--) {
    [arr[0], arr[i]] = [arr[i], arr[0]];
    heapify(arr, i, 0);
  }
  return arr;
}

function heapify(arr, size, root) {
  let largest = root;
  const left = 2 * root + 1, right = 2 * root + 2;
  if (left < size && arr[left] > arr[largest]) largest = left;
  if (right < size && arr[right] > arr[largest]) largest = right;
  if (largest ≠ root) {
    [arr[root], arr[largest]] = [arr[largest], arr[root]];
    heapify(arr, size, largest);
  }
}
```

Time: O(n log n) always. **Space:** O(1). **Stable:** No.

JAVASCRIPT Copy

```
function countingSort(arr, max) {
  const count = new Array(max + 1).fill(0);
  arr.forEach(num => count[num]++);
  let idx = 0;
  for (let val = 0; val <= max; val++) {
    while (count[val]-- > 0) arr[idx++] = val;
  }
  return arr;
}
```

Time: O(n + k). **Space:** O(k).

23.8 Radix Sort

JAVASCRIPT Copy

```
function radixSort(arr) {
  const max = Math.max(...arr);
  for (let exp = 1; Math.floor(max / exp) > 0; exp *= 10) {
    countingSortByDigit(arr, exp);
  }
  return arr;
}

function countingSortByDigit(arr, exp) {
  const output = new Array(arr.length);
  const count = new Array(10).fill(0);
  arr.forEach(num => count[Math.floor(num / exp) % 10]++);
  for (let i = 1; i < 10; i++) count[i] += count[i - 1];
  for (let i = arr.length - 1; i >= 0; i--) {
    const digit = Math.floor(arr[i] / exp) % 10;
    output[--count[digit]] = arr[i];
  }
```

23.9 Bucket Sort

Approach: Scatter values into evenly-sized buckets by their normalized position in the [min, max] range, sort each small bucket with a cheap comparison sort, then concatenate the buckets in order.

JAVASCRIPT Copy

```
function bucketSort(arr, bucketCount = 10) {
  const n = arr.length;
  if (n ≤ 1) return arr.slice();

  // Find range so this works for any numbers, not just [0, 1).
  let min = arr[0], max = arr[0];
  for (const x of arr) {
    if (x < min) min = x;
    if (x > max) max = x;
  }
  if (min === max) return arr.slice(); // all equal

  const buckets = Array.from({ length: bucketCount }, () ⇒ []);
  const range = max - min;

  // Place each value into the bucket for its slice of the range.
  for (const x of arr) {
    let idx = Math.floor(((x - min) / range) * bucketCount);
    if (idx === bucketCount) idx = bucketCount - 1; // the max value lands
    buckets[idx].push(x);
  }

  // Sort each (small) bucket, then stitch them together left to right.
  const result = [];
  for (const bucket of buckets) {
    bucket.sort((a, b) ⇒ a - b);
    for (const x of bucket) result.push(x);
  }
  return result;
```

```
// If input is uniformly distributed, buckets stay small and this runs in
console.log(bucketSort([0.42, 0.17, 0.99, 0.03, 0.5, 0.71])); // [0.03, 0.
console.log(bucketSort([29, 25, 3, 49, 9, 37]));              // [3, 9, 2
```

Distribute elements into buckets, sort each, concatenate. Best when uniformly distributed. **Time:** O(n) average. **Space:** O(n + k).

23.10 Tim Sort

Approach: Show that V8's Array.prototype.sort is Tim Sort — a stable hybrid that detects already-ordered runs and merges them — by sorting objects on one key and confirming equal keys keep their original order.

JAVASCRIPT Copy

```
// Tim Sort = hybrid of insertion sort (for small "runs") + merge sort
// (to combine runs). It scans for naturally ordered runs, extends short
// ones with insertion sort, then merges. Since ES2019 the spec REQUIRES
// Array.prototype.sort to be STABLE, and V8 implements it with Tim Sort.

const people = [
  { name: 'Ada', age: 30 },
  { name: 'Ben', age: 25 },
  { name: 'Cid', age: 30 },
  { name: 'Dot', age: 25 },
];

// Sort by age only. Ben & Dot both 25; Ada & Cid both 30.
const byAge = [...people].sort((a, b) => a.age - b.age);

// Stability: within each equal-age group the ORIGINAL order survives,
// so Ben stays before Dot and Ada stays before Cid.
console.log(byAge.map(p => p.name)); // [ 'Ben', 'Dot', 'Ada', 'Cid' ]

// A run-detection sanity check: an already-sorted array is left untouched
// which is exactly the fast path Tim Sort exploits.
```

Hybrid of merge sort and insertion sort. V8 and SpiderMonkey both use it. Stable, O(n log n) worst case. In interviews: implement sorts from scratch to show understanding; in production use `arr.sort((a, b) => a - b)`.

23.11 Custom Comparators

JAVASCRIPT Copy

```
products.sort((a, b) => a.price - b.price);                    // By
products.sort((a, b) => a.name.localeCompare(b.name));          // A
products.sort((a, b) => {                                       // M
  if (a.category !== b.category) return a.category.localeCompare(b.categor
  return a.price - b.price;
});
items.sort((a, b) => {                                          // N
  if (a.date == null) return 1;
  if (b.date == null) return -1;
  return new Date(a.date) - new Date(b.date);
});
```

23.12 Sorting Stability

Approach: Demonstrate that a stable sort preserves the pre-existing relative order among records with equal sort keys — exactly what lets a UI table re-sort by a second column without scrambling the first.

JAVASCRIPT Copy

```
// A UI table already sorted by NAME. The user now clicks the DEPT header.
// A stable sort must keep the name order WITHIN each department, so the
```

```
// table stays intuitive instead of shuffling equal rows randomly.
const rows = [
  { name: 'Amy', dept: 'Sales' },
  { name: 'Bea', dept: 'Sales' },
  { name: 'Max', dept: 'Eng' },
  { name: 'Zoe', dept: 'Eng' },
];
// (rows are already in name order within their eventual groups)

// Sort by dept only. JS's sort is stable, so equal-dept rows keep order.
const byDept = [...rows].sort((a, b) => a.dept.localeCompare(b.dept));
console.log(byDept.map(r => r.name)); // [ 'Max', 'Zoe', 'Amy', 'Bea' ]

// What an UNSTABLE sort could do instead (simulated by tie-breaking on a
// value unrelated to the original order): equal-dept rows get scrambled.
const unstable = [...rows]
  .map((r, i) => [r, i])
  .sort((a, b) => {
    const d = a[0].dept.localeCompare(b[0].dept);
    return d !== 0 ? d : b[1] - a[1]; // ties broken by REVERSED index
  })
  .map(([r]) => r.name);
console.log(unstable); // [ 'Zoe', 'Max', 'Bea', 'Amy' ] - name order lost

// Lesson: rely on the stable built-in sort, or include a stable tie-break
// in your comparator, so re-sorting a table never scrambles equal rows.
```

Equal elements maintain original relative order. User sorts table by "Name" then "Department" — with stable sort, names stay ordered within each department. JS `.sort()` is stable since ES2019.

QUICK CHECK — no marks, just for you.

1. A user sorts a table by "Name", then re-sorts the same table by "Department". With a stable sorting algorithm, what is guaranteed about the final result?

- ○ Rows with an equal department value are dropped down to the end of the table
- ○ Within each department, the rows remain in the name order from the previous sort

- The original array is left untouched and a new sorted array is returned
- The second sort runs in O(n) because the data is already partially sorted

Check answer

2. Using the lesson's "nulls last" custom comparator, what does this snippet log?

JAVASCRIPT Copy

```
const items = [
  { id: 1, date: '2024-03-01' },
  { id: 2, date: null },
  { id: 3, date: '2024-01-15' },
];

items.sort((a, b) => {
  if (a.date == null) return 1;
  if (b.date == null) return -1;
  return new Date(a.date) - new Date(b.date);
});

console.log(items.map(x => x.id).join(','));
```

- 3,1,2
- 2,3,1
- 1,3,2
- 3,2,1

Check answer

3. This is the partition helper from the lesson's quick sort. What does the snippet log?

JAVASCRIPT Copy

```
  let i = lo;
  for (let j = lo; j < hi; j++) {
    if (arr[j] < pivot) { [arr[i], arr[j]] = [arr[j], arr[i]]; i++; }
  }
  [arr[i], arr[hi]] = [arr[hi], arr[i]];
  return i;
}

const a = [5, 2, 8, 1, 4];
const p = partition(a, 0, a.length - 1);
console.log(p);
console.log(a.join(','));
```

- ○ 3
2,1,4,5,8
- ○ 2
2,1,4,8,5
- ○ 2
1,2,4,5,8
- ○ 2
2,1,4,5,8

Check answer

‹ Previous
Tree and graph structures

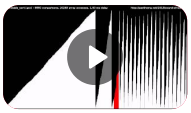
Next ›
Searching

LEARN MORE

ARTICLES & DOCS

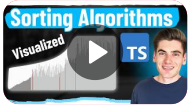
Frontend System Desi...

VIDEOS



15 sorting algorithms in 6 minutes

YouTube



5 must-know sorting algorithms

YouTube



Every sorting algorithm explained

YouTube

Hand-picked references for this lesson.